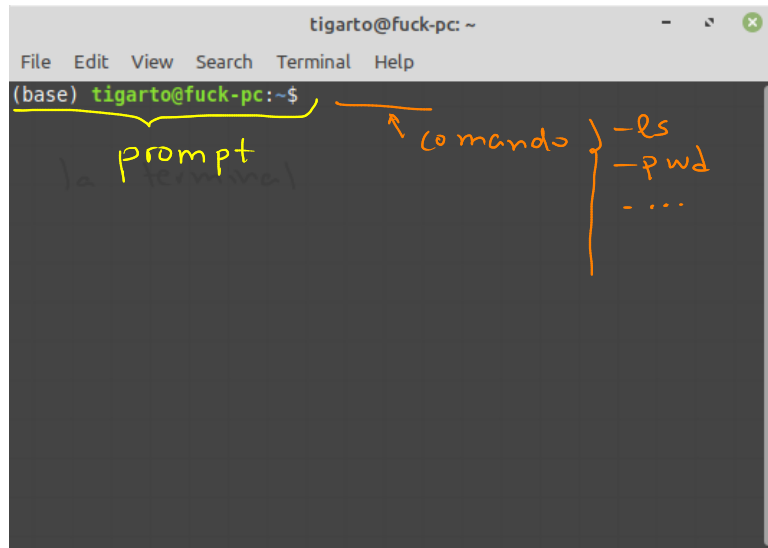


17/10/2020 - Laboratorio de Procesos (Shell).

Agenda.

- Shell: Imitar la terminal



- Comandos de compilación:

gcc $\cup$ wish.c $\cup$ -o wish $\cup$ -Wall $\cup$ -Werror

- Sobre el shell

a. Modos de funcionamiento

i. Modo interactivo (espera a que el usuario ingrese un comando).

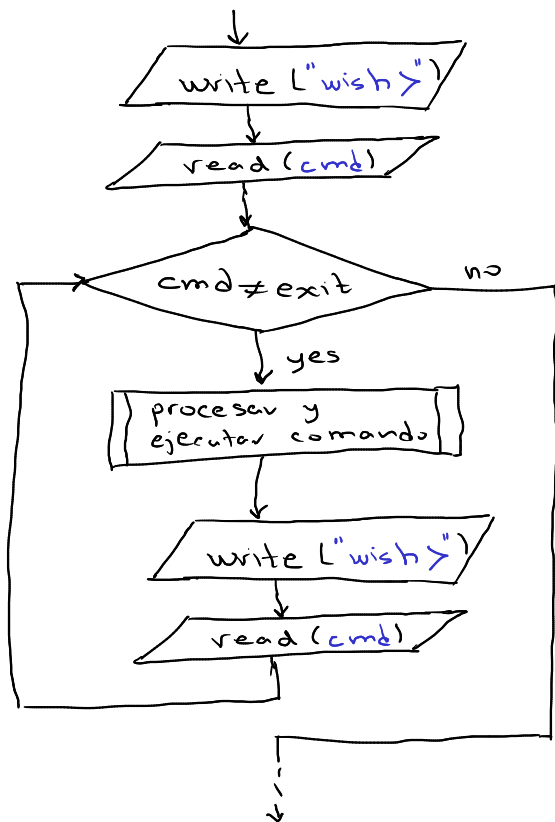
```
prompt> ./wish
wish>
```

ii. Modo batch (Se pasan comandos a través de un archivo batch. Algo similar a un script))

```
prompt> ./wish batch.txt
```

b. Estructura.

- Ciclo infinito -



```
const char *mypath[] = {
    "./",
    "/usr/bin/",
    "/bin/",
    NULL
};

/** Dentro del main del intérprete */
while (...) {
    /* Wait for input */
    printf ("prompt> ");
    fgets (...);
    /* Parse input */
    while (( ... = strsep (...) ) != NULL) {
        ...
    }

    /* If necessary locate executable using mypath array */
    /* Launch executable */
    if (fork () == 0) {
        ...
        execv (...);
        ...
    }
    else
    {
        wait (...);
    }
}
```

¿Como sería el ciclo por dentro?

1. Con el comando `exit` se sale del shell `wish`.

2. Leer el comando con `getline()` o similar.

```
const char *mypath[] = {
    "./",
    "/usr/bin/",
    "/bin/",
    NULL
};

/** Dentro del main del intérprete */
while (...) {
    /* Wait for input */
    printf ("prompt> ");
    fgets (...);
    /* Parse input */
    while (( ... = strsep (...) ) != NULL) {
        ...
    }

    /* If necessary locate executable using mypath array */
    /* Launch executable */
    if (fork () == 0) {
        ...
        execv (...);
        ...
    }
    else
    {
        wait (...);
    }
}
```

Impresión del prompt.

② Lectura del comando: `getline()` o similar

3. Procesar el comando (Obtener cada uno de sus componentes) Usar `strsep()` o similar.

```
const char *mypath[] = {
    "./",
    "/usr/bin/",
    "/bin/",
    NULL
};

/** Dentro del main del intérprete */
while (...) {
    /* Wait for input */
    printf ("prompt> ");
    fgets (...);
    /* Parse input */
    while ((... = strsep (...)) != NULL) {
        ...
    }

    /* If necessary locate executable using mypath array */
    /* Launch executable */
    if (fork () == 0) {
        ...
        execv (...);
        ...
    }
    else
    {
        wait (...);
    }
}
```

Consultar ^{varios} ejemplos.
Buscar

3

wish> ls -ahl /tmp

Comando completo.

partes

"ls"
"-ahl"
"/tmp"
'\0'

Vemos que hay bastante similitud con el `argc` y el `argv`.

4. Ejecutar el comando:

Un comando puede ser interno o externo.

* Caso de un comando externo:

- Un comando externo lo hizo otro y nos dio su ejecutable.

tigarto@fuck-pc: /bin

File Edit View Search Terminal Help

(base) tigarto@fuck-pc:/bin\$ ls

archdetect df ln ntfssecaudit systemd-hwdb
bash dir loadkeys ntfstruncate systemd-inhibit
brltty dmesg login ntfsusermap systemd-machine-id-setup
btrfs dnsdomainname loginitctl ntfswipe systemd-notify
btrfsck domainname lowntfs-3g open systemd-sysusers
btrfs-debug-tree dumpkeys ls openvt systemd-tmpfiles
btrfs-find-root echo lsblk pidof systemd-tty-ask-password-agent
btrfs-image ed lsmod ping tar
btrfs-map-logical efibootdump mkdir ping4 tempfile
btrfs-select-super efibootmgr mkfs.btrfs ping6 touch
btrfsstune egrep mknod plymouth true
btrfs-zero-log false mktemp ps udevadm
bunzip2 fgconsole more pwd ulockmgr_server
busybox fgrep mount rbash umount
bzipcat findmnt mountpoint readlink uname

Comando ls

En este directorio se encuentran los siguientes ejecutables

- los comandos externos se llaman usando funciones como `fork()`, `exec()`, `wait()` / `waitpid()`.

Ejemplo:

- Ejecucion en consola

wish > ls -ahl /tmp

- Diagrama de llamadas
(Abajo la forma basica del
codigo)

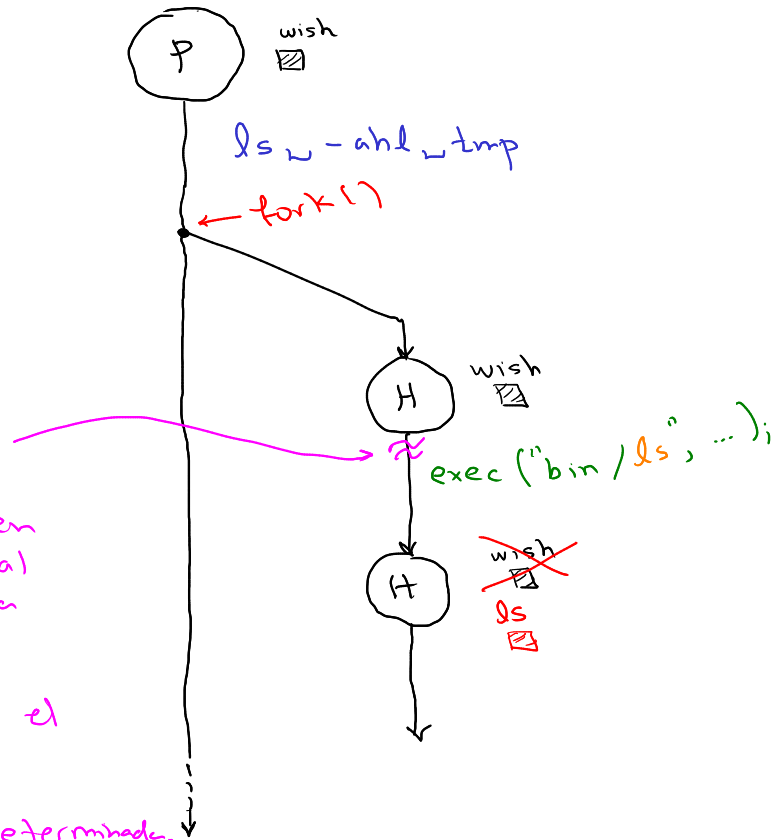
Sobre los paths

Importante:

- i. Un ejecutable se encuentra en un directorio al cual se asocia una ruta.
- ii. Es necesario determinar si el ejecutable se encuentra en una ruta determinada.



Entonces - Que es eso de las rutas?



A continuación se muestra el esqueleto de como llamar un comando externo.

Forma basica

Harev uso de las parejas fork()/exec().

```
/** Dentro del main del interprete */
...
/* Hacer el parsing de la entrada */
num = separaItems (expresion, &items, &background);
...
/* Obtener comando */
if(ordenaIngresada == orden1) {
    /* Lanzar el ejecutable asociado a la orden 1 */
    //Codigo...
}
...
else if(ordenaIngresada == ordenN) {
    /* Lanzar el ejecutable asociado a la orden 1 */
    //Codigo: suponiendo que es externa...
    if (fork () == 0) {
        ...
        execv (...); // Acá va la invocación de la orden externa
        ...
    }
    ...
}
...
```

* Variable PATH - path

- El path es una lista ordenada de nombres de rutas que especifican los directorios donde el shell debera buscar los comandos externos.

Por ejemplo suponiendo que la variable path cuenta con las rutas: "/bin", "/usr/bin". Tenemos la siguiente declaracion en C:

```
char *PATH[] = {
    "/bin",
    "/usr/bin",
    NULL};
```

PATH	
"/bin"	
"/usr/bin"	
NULL	

* Antes de llamar el exec(...) es necesario que el comando se encuentre en alguna de las rutas del PATH.

* La verificación sobre si un ejecutable se encuentra en un directorio, se hace con la función access

wish > ls -ahlw /mnt

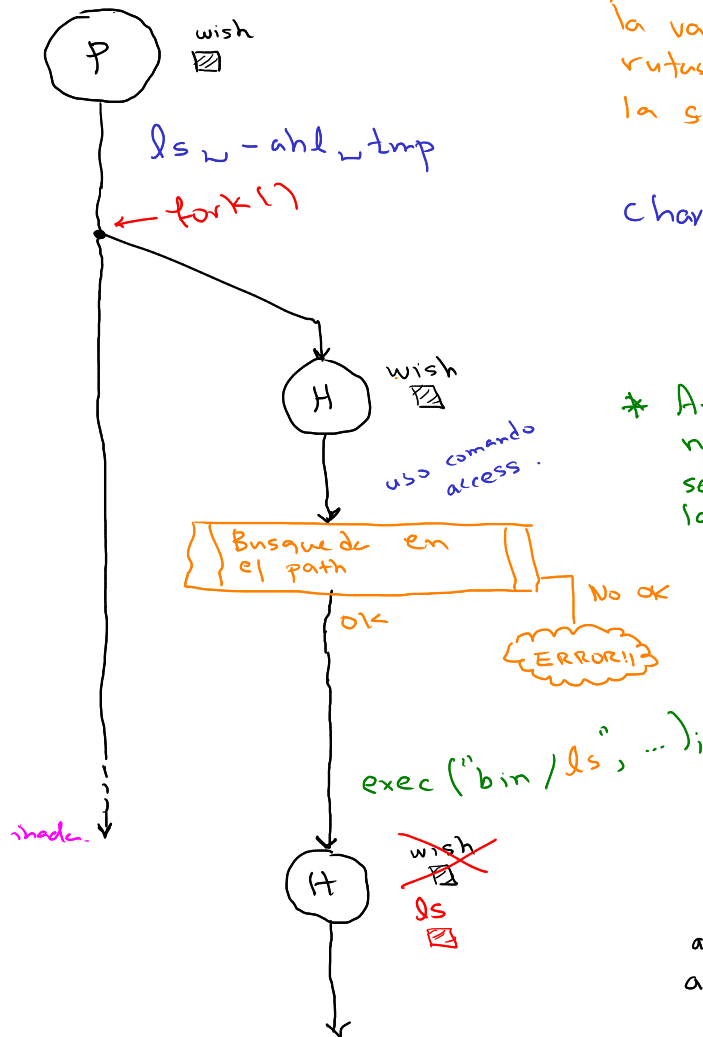
```
access("/bin/ls", X_OK); →
access("/usr/bin/ls", X_OK); →
... →
```

PATH

"/bin"

"/usr/bin"

NULL



* Comandos integrados

- Son funciones que ejecutan llamadas al sistema (cuando sea necesario) y otras instrucciones; con el fin, de realizar una tarea determinada.
- Para esta practica.

comando	Llamada a invocar
exit	exit
cd	chdir
path *	---

* Esta funcion lo que hace es modificar la variable PATH.

Más sobre el path

comando	contenido del PATH
path = /bin /usr/bin	"/bin" "/usr/bin" '\0'
path = /bin .	"/bin" "." '\0'
path =	'\0'

→ No se logra ejecutar ningún comando externo.

Comandos integrados - Código esqueleto

Funciones asociadas a los comandos integrados

Invocación de un comando integrado

```

/** Comandos como funciones */
tipo_retorno orden1 (tipo args, ...);
...
tipo_retorno ordenN(tipo args, ...);

/** Dentro del main del intérprete*/
...
/* Hacer el parsing de la entrada */
num = separaItems (expresion, &items, &background);
...
/* Obtener comando */
if(ordenIngresada == orden1) {
    /* Lanzar el ejecutable asociado a la orden 1 */
    // Código...
}
...
else if(ordenIngresada == ordenN) {
    /* Lanzar el ejecutable asociado a la orden 1 */
    // Código: suponiendo que es interna...
    ordenN(parametros); // Como se llame dependerá si tiene o no &
}
...

```

→ Pero demos que un comando integrado es una función.

5. Redirección.

- Redirección de salida estándar (>).

wish>ls _ahl_tmp

~~~~~  
~~~~~  
~~~~~

wish>

wish> ls \_ahl\_tmp > output  
wish>

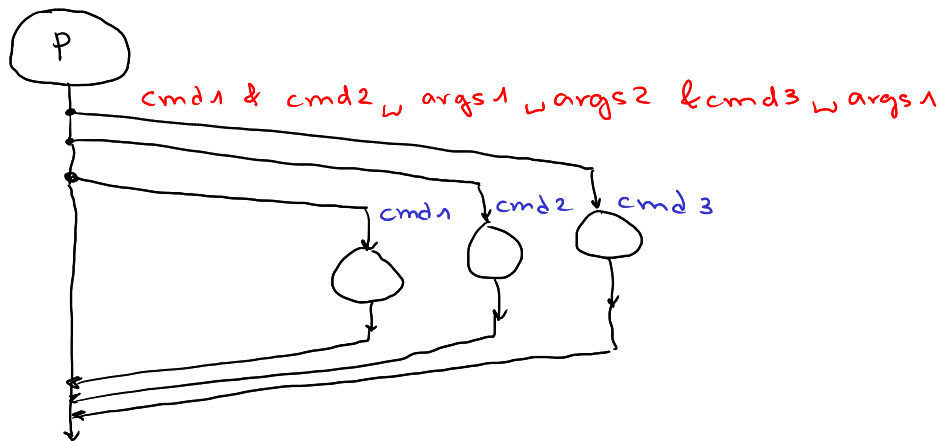


\*Nota: No hay redirección de los comandos integrados.

## 6. Comandos paralelos.

- Uso del comando ampersand (&).
- Clave: Use wait().

wish> cmd1 & cmd2 \_args1 \_args2 &cmd3 \_args1



## 7. Errores.

- Solo hay un mensaje de error.

```
char error_message[30] = "An error has occurred\n";  
write(STDERR_FILENO, error_message, strlen(error_message));
```

An error has occurred.